excel-1710-2020-assignments - pp07.pyxls

Problem set 7. Review: Database Access & Exponential Growth

Too many people spent far too much time on PS03.pyxls

Therefore we are going to back up and review how to pull numbers out of databases, and we are going to review exponential growth processes.

Let us get started!

1. Preliminaries

A. Computing environment

First, let us set up our computing environment with the libraries we need:

```
In [ ]: # J.I.A.D. set up the computing environment: ensure that graphs
        # appear inline in the notebook and in extra windows:
        %matplotlib inline

        # J.I.A.D. set up the computing environment: import other libraries

        import numpy as np
        import pandas as pd
        import matplotlib as mpl
        import matplotlib.pyplot as plt
```

B. Reproduce our very long-run growth dataframes for the world, the global north, and the global south

Recall our estimates of humanity's economy and its growth over the long Left's repeat their construction:

```
In [ ]: # J.I.A.D.: reproducing the creation of the world dataframe:

        long_run_growth_list = [
            [-10000, 0.1, 1.100, 170,47],
            [-9000, 0.1, 1.100, 170,47],
            [-8000, 0.1, 1.100, 170,47],
            [-7000, 0.1, 1.100, 170,47],
            [-6000, 0.1, 1.100, 170,47],
            [-5000, 0.1, 1.100, 170,47],
            [-4000, 0.1, 1.100, 170,47],
            [-3000, 0.1, 1.100, 170,47],
            [-2000, 0.1, 1.100, 170,47],
            [-1000, 0.1, 1.100, 170,47],
            [0, 0.1, 1.100, 170,47],
            [1000, 0.1, 1.100, 170,47],
            [1500, 0.1, 1.100, 170,47]]

        long_run_growth_df = pd.DataFrame(
            long_run_growth_list, columns = ['year', 'population',
            'income_level', 'human_index'])

        long_run_growth_df['year'] = long_run_growth_df['year'].astype(pd.int64)

        initial_year = long_run_growth_df['year'][0:10]

        name = 0
        for i in
        initial_year:
            name = name + 1
            print(name)

        for i in range(0,11):
            long_run_growth_df['name'][i:i+1] = long_run_growth_df['year'][i:i]
            long_run_growth_df['income_level'][i:i+1] = long_run_growth_df['income_level'][i:i]
            for j in range(int(long_run_growth_df['year'][i:i]),int(long_run_growth_df['year'][i:i+1])):
                long_run_growth_df['population'][j:j+1] = long_run_growth_df['population'][i:i]
                long_run_growth_df['income_level'][j:j+1] = long_run_growth_df['income_level'][i:i]
                long_run_growth_df['human_index'][j:j+1] = long_run_growth_df['human_index'][i:i]

        For clarity, and to note to the end of each observation, remember
        a set of what was going on in human history back in each of the
        years contained in our dataset:

        area = {'year': [0, 500, 1000, 1500],
                'description': ['Agriculture & herding', 'proto-agrarian age',
                'industrial revolution', 'modern economic growth', 'whatever the 21st century bring'],
                'industrial revolution': [0, 0, 1, 1],
                'modern economic growth': [0, 0, 0, 1]}

        long_run_growth_df['area'] = area

        format_dict = {'income_index': '{0:.0f}',
                       'income_level': '{0:.0f}', 'population': '{0:.2f}'}

        print('WORLD LEVELS')
        long_run_growth_df.style.format(format_dict)

In [ ]: # J.I.A.D.: reproducing the creation of the global north dataframe:

        long_run_growth_list_global_north = [
            [-10000, 0.1, 1.100, 170,47, 0.0001],
            [-9000, 0.1, 1.100, 170,47, 0.0001],
            [-8000, 0.1, 1.100, 170,47, 0.0001],
            [-7000, 0.1, 1.100, 170,47, 0.0001],
            [-6000, 0.1, 1.100, 170,47, 0.0001],
            [-5000, 0.1, 1.100, 170,47, 0.0001],
            [-4000, 0.1, 1.100, 170,47, 0.0001],
            [-3000, 0.1, 1.100, 170,47, 0.0001],
            [-2000, 0.1, 1.100, 170,47, 0.0001],
            [-1000, 0.1, 1.100, 170,47, 0.0001],
            [0, 0.1, 1.100, 170,47, 0.0001],
            [1000, 0.1, 1.100, 170,47, 0.0001],
            [1500, 0.1, 1.100, 170,47, 0.0001]]

        long_run_growth_df_global_north = pd.DataFrame(
            long_run_growth_list_global_north, columns = ['year', 'population',
            'income_level', 'human_index', 'relative'])

        long_run_growth_df_global_north['year'] = long_run_growth_df_global_north['year'].astype(
        pd64)

        area = {'year': [0, 500, 1000, 1500],
                'description': ['Agriculture & herding', 'proto-agrarian age',
                'industrial revolution', 'modern economic growth', 'whatever the 21st century bring
        ],
                'industrial revolution': [0, 0, 1, 1],
                'modern economic growth': [0, 0, 0, 1]}

        long_run_growth_df_global_north['area'] = area

        format_dict = {'income_index': '{0:.0f}',
                       'income_level': '{0:.0f}', 'population': '{0:.2f}', 'relative': '{0:.3f}'}

        print('GLOBAL NORTH LEVELS')
        long_run_growth_df_global_north.style.format(format_dict)

In [ ]: # J.I.A.D.: reproducing the creation of the global south dataframe:

        long_run_growth_list_global_south = [
            [-10000, 0.1, 1.100, 170,47, 0.9999],
            [-9000, 0.1, 1.100, 170,47, 0.9999],
            [-8000, 0.1, 1.100, 170,47, 0.9999],
            [-7000, 0.1, 1.100, 170,47, 0.9999],
            [-6000, 0.1, 1.100, 170,47, 0.9999],
            [-5000, 0.1, 1.100, 170,47, 0.9999],
            [-4000, 0.1, 1.100, 170,47, 0.9999],
            [-3000, 0.1, 1.100, 170,47, 0.9999],
            [-2000, 0.1, 1.100, 170,47, 0.9999],
            [-1000, 0.1, 1.100, 170,47, 0.9999],
            [0, 0.1, 1.100, 170,47, 0.9999],
            [1000, 0.1, 1.100, 170,47, 0.9999],
            [1500, 0.1, 1.100, 170,47, 0.9999]]

        long_run_growth_df_global_south = pd.DataFrame(
            long_run_growth_list_global_south, columns = ['year', 'population',
            'income_level', 'human_index', 'relative'])

        long_run_growth_df_global_south['year'] = long_run_growth_df_global_south['year'].astype(
        pd64)

        area = {'year': [0, 500, 1000, 1500],
                'description': ['Agriculture & herding', 'proto-agrarian age',
                'industrial revolution', 'modern economic growth', 'whatever the 21st century bring
        ],
                'industrial revolution': [0, 0, 1, 1],
                'modern economic growth': [0, 0, 0, 1]}

        long_run_growth_df_global_south['area'] = area

        format_dict = {'income_index': '{0:.0f}',
                       'income_level': '{0:.0f}', 'population': '{0:.2f}', 'relative': '{0:.3f}'}

        print('GLOBAL SOUTH LEVELS')
        long_run_growth_df_global_south.style.format(format_dict)
```

2. Pulling numbers out of databases

Suppose we want to pull a number out of a database—say the 1700 average income level in the global south in 2020. How do we do this? The best code will provide an example:

```
In [ ]: # J.I.A.D.: pulling out global north average income in 2020

        long_run_growth_global_north_df['income_level'][15]
```

That is all there is to it. It is a three-step process:

1. Tell the python interpreter what dataframe the number is in: "long_run_growth_global_south_df"
2. Tell the python interpreter what column the number is in: "income_level", by stating the column's name as a string inside brackets: "['income_level']"
3. Tell the python interpreter the observation—the row—the number belongs to by stating the row number inside brackets: "[15]"

Yes, we, and you, have to keep track of the fact that each year in our dataset is stored in a row, and that the number that cannot go inside quotation marks—for doing the wrong thing with either and generate an error!—There is no good answer.

Now let us get rid of the quotation marks. This should be, if it has passed the education right, be trivial for when more than half the states, and nearly double for the rest.

Place in the next cell, which has code to pull out the year 1500 value of income per capita for the world as a whole, assign it to variable "world_income_1500" and then print it out:

```
In [ ]: # J.I.A.D.: pulling out world average income in 1500

        world_income_1500 = ...
        print("global north income_1500", world_income_1500)
```

Did the code pull above print, below itself, "9872.0 is the average income per capita in the world in 1500"? If not, go back and try to figure what you do wrong. If yes, proceed.

Now can you pull out the value of the ideas stock deployed in the global north in 1870? Do so in the code set below:

```
In [ ]: # J.I.A.D.: pulling out global north ideas stock in 1870

        global_north_ideas_1870 = ...
        print(global_north_ideas_1870, "is the level of ideas used by the global north in 1870")
```

Did you succeed in pulling out 0.0001? Good. Now, in the next cell, assign the variable "relative_ideas_1870" to be the number in the 6th row of the "human_index" column in the "long_run_growth_global_south" dataframe. You can do anything with it that you can do with a number. In particular, you can do calculations with it. In particular, you can divide it by another number.

So in the code cell below, do so, calculate what the value of the relative ideas stock in the global north was relative to those deployed in the global south in year 1870 by dividing the global north ideas value by the global south ideas value:

```
In [ ]: # J.I.A.D.: global north ideas divided by global south ideas in year 1

        north_south_ideas_ratio_1 = ...
        print(north_south_ideas_ratio_1, "is the north-south ratio of the value of ideas in year 1")
```

Now let us report these differences in the four code cells below, for the average calculations for the years 1500, 1770, 1870, and 2020 to see how the region that was to become the heart of the global north developed a disproportionate technology edge over the rest of the world between the year 1 and today:

```
In [ ]: # J.I.A.D.: global north ideas divided by global south ideas in year 1500

        north_south_ideas_ratio_1500 = ...
        print(north_south_ideas_ratio_1500, "is the north-south ratio of the value of ideas in 1500")

In [ ]: # J.I.A.D.: global north ideas divided by global south ideas in year 1770

        north_south_ideas_ratio_1770 = ...
        print(north_south_ideas_ratio_1770, "is the north-south ratio of the value of ideas in 1770")

In [ ]: # J.I.A.D.: global north ideas divided by global south ideas in year 1870

        north_south_ideas_ratio_1870 = ...
        print(north_south_ideas_ratio_1870, "is the north-south ratio of the value of ideas in 1870")

In [ ]: # J.I.A.D.: global north ideas divided by global south ideas in year 2020

        north_south_ideas_ratio_2020 = ...
        print(north_south_ideas_ratio_2020, "is the north-south ratio of the value of ideas in 2020")
```

By now you should have five numbers, covering the period from year 1 to 2020 (0.00, 1.25, 1.45, 2.88, 5.14).

What would happen if you tried to divide not one number but another number, but divide one column from a dataframe by another column from another dataframe? Let's see:

```
In [ ]: # J.I.A.D.: dividing one series by another element-by-element

        m_a_ideas_ratio = long_run_growth_global_north_df['human_index']/long_run_growth_global_south_df['human_index']

        print(m_a_ideas_ratio, "is the ideas-ratio series")
```

We now have all of the ideas-ratio quotients from the year -10000 to today in a series, where we can refer to all them together, or pick out one in the row-column set, just from the ideas-ratio for the year -5000:

```
In [ ]: # J.I.A.D.: picking out the ideas-ratio value for the year -5000

        m_a_ideas_ratio_minus_5000 = ...
        print(m_a_ideas_ratio_minus_5000, "is the ideas ratio for the year -5000")
```

In dividing two columns from different dataframes, we have lost track of the year variable—it is no longer included and attached. So we pull back by making a big dataframe consisting of our "which year this or series and our ideas-ratio series by using pandas concat:

```
In [ ]: # J.I.A.D.: creating a big dataframe

        m_a_ideas_ratio_df = pd.concat([long_run_growth_global_north_df['year'], m_a_ideas_ratio],
        axis=1)

        m_a_ideas_ratio_df
```

Note that it might be clearer what we are doing if, instead of carrying the index row number 0 (0 around, we used the "year" column as an index field:

```
In [ ]: # J.I.A.D.: changing the index to the 'year' column

        m_a_ideas_ratio_df.set_index('year', inplace=True)

        m_a_ideas_ratio_df
```

Then, when we wanted to refer to the ideas ratio for the year -4000, we would not have to remember which row of the table corresponds to the year -4000, but simply specify that it output of the row "-4000":

```
In [ ]: # J.I.A.D.: referring to entries by year rather than by row number

        m_a_ideas_ratio_df['ratio'][-4000]
```

A problem is that we then have the ability to access the year column as a series of numbers, since it is now the index:

```
In [ ]: # J.I.A.D.: generating an error message

        m_a_ideas_ratio_df['year'][-4000]
```

We have just generated an error: the python interpreter could not find a database column variable named 'year' because we transformed it into an index.

If we want to have the year lying around in the dataframe so that we can do math with it, we have to add it back in:

```
In [ ]: # J.I.A.D.: adding the year back in

        m_a_ideas_ratio_df['year'] = 0.0000, -0000, -0000, -0000, -0000, 1, 1500, 1770, 1870, 2020
```

3. Exponential growth

A number of problems in the problem set concern the 3 calculating growth rates. So here is some math review in preface—and a review of the rules of exponents. Let us first review what we know for either cases that we have, and apply digits for the rest.

Let us start with a quantity X equal to 1. In our initial year—call it year 0, $X_0 = 1$. Suppose that each year it grows by 1%. Then when we move from year 0 to year 1, we multiply the year's value by 1.01 to get the year 1's value: $X_{year} = 1.01 \cdot X_0$.

What then will the value of X be in the 10th year? What is X_{10}?

Step one:

$X_1 = 1$
$X_2 = (1.01)X_1 = 1.01$
$X_3 = (1.01)X_2 = (1.01)^2X_1 = 1.0201$
$X_4 = (1.01)X_3 = (1.01)^3X_1 = 1.030301$
$X_5 = (1.01)X_4 = (1.01)^4X_1 = (1.01)^4X_1 = 1.04060401$
$X_6 = (1.01)X_5 = (1.01)^5X_1 = (1.01)^5X_1 = 1.0510100501$

and so, multiplying by a factor 1.01 each year. This is the list of thing companies are made for:

```
In [ ]: # J.I.A.D.: looping to calculate 1.01^n:

        for i in range(1,11):
            x = 1.01*x
            print(x, i)
```

It takes a ten under 10 years for a quantity growing exponentially at the rate of 1% per year to double. If we write 1 for the doubling time and subscript it by the growth rate, we have:

$T_{1\%} = 70$

In general, if we have a quantity X which starts at our baseline time—call it time zero—and it grows at a rate g, then after t years, it will be:

$X_t = X_0(1+g)^t$

and if we only can express time in logarithms—which turn exponentiation into multiplication, and multiplication into addition, then:

$\ln(X_t) = \ln(X_0) + t\ln(1+g)$

Let's go back to doubling times.

Now suppose that we wanted to find out how long a time T it takes a quantity growing at some other rate—2% or 3% or 5% or 20% per—to double. We would try trial and error for each growth rate. We could state of the equation:

$2 = (1+g)^T$

and try to figure out how to solve it.

It is much simpler to solve if we look at the logarithmic form:

$\ln(2) = \ln(1+g)T$

and then simply solve:

$T = \frac{\ln(2)}{\ln(1+g)}$

We know that $\ln(2) = 0.6931471$. And we also know that $\ln(1+g)$ will be very close to g for small g. So:

```
In [ ]: # J.I.A.D.: ln(1+x)/x for x = 0.01

        g = 0.01

        print("g", ...)print("ln(1+g)", ...)print("ln(1+g)/g", ...)
        print("g, ln(1+g), ln(1+g)/g", ...)print("g", ...)print("ln(1+g)", ...)
```

Very, very close, no? The difference between .01 and .00995 is only one part in 200. For g=10% the difference between g and ln(1+g) is only one part in 20. For g=20% it is one part in 10. For g=30% it is one part in five. Only when we get to 50% growth do we have a 10% difference between g and ln(1+g)—a fairly good approximation. Even at 100% growth rate of less than 10% (you are worried, I understand) still of one part deviation at 1 in 0.7, or 1.4.

So what is the rule of thumb for the time it takes for a growth rate of 1% per year? We can use the exact equation: $T_{1\%} = \frac{\ln(2)}{\ln(1+.01)} \approx \frac{\ln(2)}{.01}$ or 1%/year. Do the calculation in the next code cell, remembering that "ln(2)" is the pandas log function:

```
In [ ]: # J.I.A.D.: ln(2)/.01: the time to point out the same thing as the rule of 70, and yet the proportional difference between g and ln(1+g)—for a passing nature from 1% to 2% to 5% and on up to 10%:
```

4. J.I.A.D.: exact doubling time for 1%/year growth rate

```
In [ ]: exact_doubling_time_for_1_percent = ...

        print(exact_doubling_time_for_1_percent, "is the exact doubling time for 1%/year growth r
        ate")
```

00.6931 years, no?

Now program us a new table to calculate exact doubling times for a equal to 1%, 2%, 3%, and so forth up to 10%/year:

```
In [ ]: # J.I.A.D.: exact doubling times for g from 1% to 10%/year

        # for i in range(...):
        # g = ...
        # T = ...
        # print("years to the actual doubling time for an annual growth rate of", i, "%/year"
        ...)

In [ ]: for i in range(1,11): # delete before publishing
            g = 0.01*i # delete before publishing
            T = ...log(2)/...log(...) # delete before publishing
            print("years to the actual doubling time for an annual growth rate of", i, "%/year"
            ) # delete before publishing
```

Note that for a growth rate of 9% per year the product of the growth rate and the doubling time is:

```
In [ ]: # J.I.A.D.: doubling time multiplied by growth rate

        ...log(2)/...log(1+0.09)
```

In fact, for all but the growth rates, if you multiply the growth rate in percent by the doubling time you get:

```
In [ ]: # J.I.A.D.: doubling times multiplied by growth rates

        for i in range(...):
            ...
            print(...*...)
```

Hence the rule of thumb called the **Rule of 70**: Want to know how long it takes a quantity growing exponentially to double? Take the number 70, and divide by the growth rate in percent per year. That is, approximately, how long it takes to double.

Conceptually, if you have a quantity X that starts in the year 1500 at a value of $X_{year} = 1.11$ and that is thereafter growing at an annual rate of 0.1667%/year, you can calculate its value X_{2020} in the current year using the equation:

$X_t = X_0(1+g)^t$

Where T, the number of years for which it grows, is 520, and where the growth rate g = 0.1667%/year to get:

```
In [ ]: # J.I.A.D.: calculating the answer to 4 in problem set 7

        X_2020 = 1.11 * 1.0016679^520

        X_2020
```

That was supposed to be the answer to 3.I.C.6.d: in problem set 7, in the year 2,020 GNP growth rate calculated in part ab, and the 520 years between 1500 and 2020. Yet many fewer people get that answer than should have.

And if you have a quantity that has a value X_0 in some baseline year 0 and a value X_T T years later, you can calculate the growth rate over that span using the equation, approximately:

$g = \frac{X_T - X_0}{X_0}$

or, exactly:

$\ln(1+g) = \frac{\ln(X_T) - \ln(X_0)}{T}$

5. Ungraded!

Now, for a final, extra, ungraded part of the problem set, recall the evolution over time of the north-south ratio of the value of their ideas stocks that we calculated at the end of part of this problem set:

```
In [ ]: # J.I.A.D.: reviewing the north-south ideas ratio

        ...
        ...
        ...
        ...
        ...
```

Calculate and stuff into a list called "growth" the difference between the global north and the global south in the growth rate of the ratio of the value of ideas stocks in the 500 years between 1500 and 2020. Hint: the first number in the list will be zero. Hint on Hint: it matters how many people can simply write down a correct answer quickly, and how many cannot.

```
In [ ]: # J.I.A.D.: calculate the difference in ideas stocks

        data = 0.0000, -0000, -0000, -0000, 1, 1500, 1770, 1870, 2020
        ...
        ...

In [ ]: for i in range(1,...):
            growth = growth + ((X_i_ideas_ratio_df['ratio'][i+1]/X_i_ideas_ratio_df['ratio'][i
            ])/...)/...
            print("ideas difference growth rate", i, growth)
```

Hint: Let me leave you with one last piece of information that you may find useful if it takes 7 years for a quantity to double. It takes only ten times as long—10 years—for it to multiply a thousandfold. :)

```
In [ ]: # summary calculations ran:

        print(world_income_1500, "is the average income per capita in the world in 1500")
        print(north_south_ideas_ratio_1500, "is the north-south ratio of the value of ideas in 1500
        ")
        print(north_south_ideas_ratio_1, "is the north-south ratio of the value of ideas in year 1
        ")
        print(north_south_ideas_ratio_1770, "is the north-south ratio of the value of ideas in 177
        0")
        print(north_south_ideas_ratio_1870, "is the north-south ratio of the value of ideas in 187
        0")
        print(north_south_ideas_ratio_2020, "is the north-south ratio of the value of ideas in 202
        0")
        print(exact_doubling_time_for_1_percent, "is the exact doubling time for 1%/year growth r
        ate")
        print("average doubling time for 1%/year growth rate", ...)
        print("average doubling time for 2%/year growth rate", ...)
        print("average doubling time for 3%/year growth rate", ...)
        print("average doubling time for 4%/year growth rate", ...)
        print("average doubling time for 5%/year growth rate", ...)
        print("average doubling time for 6%/year growth rate", ...)
        print("average doubling time for 7%/year growth rate", ...)
        print("average doubling time for 8%/year growth rate", ...)
        print("average doubling time for 9%/year growth rate", ...)
        print("average doubling time for 10%/year growth rate", ...)
        print("average doubling time for 1%/year growth rate", ...)
        print("average doubling time for 2%/year growth rate", ...)
        print("average doubling time for 3%/year growth rate", ...)
        print("average doubling time for 4%/year growth rate", ...)
        print("average doubling time for 5%/year growth rate", ...)
        print("average doubling time for 6%/year growth rate", ...)
        print("average doubling time for 7%/year growth rate", ...)
        print("average doubling time for 8%/year growth rate", ...)
        print("average doubling time for 9%/year growth rate", ...)
        print("average doubling time for 10%/year growth rate", ...)
        print("average doubling time for 1%/year growth rate", ...)
        print("average doubling time for 2%/year growth rate", ...)
        print("average doubling time for 3%/year growth rate", ...)
        print("average doubling time for 4%/year growth rate", ...)
        print("average doubling time for 5%/year growth rate", ...)
        print("average doubling time for 6%/year growth rate", ...)
        print("average doubling time for 7%/year growth rate", ...)
        print("average doubling time for 8%/year growth rate", ...)
        print("average doubling time for 9%/year growth rate", ...)
        print("average doubling time for 10%/year growth rate", ...)
        print("average doubling time for 1%/year growth rate", ...)
        print("average doubling time for 2%/year growth rate", ...)
        print("average doubling time for 3%/year growth rate", ...)
        print("average doubling time for 4%/year growth rate", ...)
        print("average doubling time for 5%/year growth rate", ...)
        print("average doubling time for 6%/year growth rate", ...)
        print("average doubling time for 7%/year growth rate", ...)
        print("average doubling time for 8%/year growth rate", ...)
        print("average doubling time for 9%/year growth rate", ...)
        print("average doubling time for 10%/year growth rate", ...)
```

6. Done!

Print your finished notebook to pdf, and upload it as an answer on the problem set 7 assignment page. LIFO.

8. Appendix Programming Dos and Don'ts...

Do:

1. **Do** make your normal run code up to your current working point every time you begin a new session. Or, if you are a little time. But if you don't, sooner or later the machine's namespace will get confused, and then you will get confused about the state of the machine's namespace, and by assuming things about what the data and files you still have lours and hours.
2. **Do** ensure that page when restarting the kernel does not seem to do the job.
3. **Do** add code cells by copying them below your current version and then working on the copy: when you break everything in the current cell you can go back to the one you had just to fire old stuff and start fresh.
4. **Do** exercise ugly development practices if there is line of code that you have found and fixed, but it. The best way to fear is to use the machine to echo back to you the thing you have just created in the namespace to make sure that it is what you meant it to do. Only after you are **sicker** that your namespace contains what you think it does should you write the line of code into your notebook and then run it to make sure it still works.
5. **Do** take screenshots of your error messages...
6. **Do** google error messages. Someone else has seen your dead friend here...
7. **Do** not confuse assignment ("=") and test for equality ("=="). In general, if there is an "if" anywhere nearby, you should be looking for equality. If there is not, you should be assignment. Equality is almost always in conditional or loop expressions. Also, **Do** note that the machine will not, in general, tell you that you have confused assignment and equality. Assignment is not two different and not compatible symbols for equals as development and equals as test.

Thanks for: Rachel Gonzalez, Christopher Hench, Stephanie Hurwitz, Sarah Jacobs, Sofiya Ksenofontova, Gabriel Lozano, Ronald Wallace...

https://tinyurl.com/20190119a-delong

This website does not host notebooks, it only renders notebooks available on other websites.

Delivered by Fastly, Rendered by OVHcloud

nbviewer GitHub repository.

nbviewer version: cf693bd

nbconvert version: 5.6.1

Rendered 2 minutes ago